HANDS-ON WORKSHOP

Fraud Detection Workshop

Train a credit card fraud model as a Domino Job, inspect and register it, publish it as a REST endpoint, then put a web app in front of it. Then add two open-source LLM agents on a GPU: one that simulates transactions from a chat, and one fine-tuned to score them for fraud. Every step is done in the Domino web interface, in a Domino-hosted project. An administrator prepares the compute environments and hardware tiers beforehand (admin setup guide), so you only select them.

TRANSACTIONS	INPUT COLUMNS	LABELED FRAUD	MODEL
78,324	**18**	**36%**	**XGBoost**
77,210 after cleaning	plus a Class label	27,952 of 77,210	100 trees, depth 4

The path at a glance

The order below is the order you will work in. A model can only be registered from a finished training run, so the training Job comes before registration. The two data steps come first because the training script reads a file they create.

STEP	YOU DO	YOU END UP WITH
1	Create a project from Git or a zip, add the data, pick the environment	A project that can run every later step
2	Run a notebook, then a Job	`clean_cc_transactions.csv` and `transformed_cc_transactions.csv`
3	Run the XGBoost training script as a Job	A tracked experiment run with metrics, plots and a model
4	Read the Job output and the Experiments page	A view of how good the model is
5	Register the model from its run	A named, versioned model in the registry
6	Publish that version as a model endpoint	A REST URL and an access token

STEP	YOU DO	YOU END UP WITH
7	Set two variables, then publish the Streamlit app	A web page that scores a transaction
8	Submit a transaction and confirm the answer came from the model	An end-to-end result
9	Fine-tune a fraud LLM as a GPU Job, then redeploy the app on a GPU with the second tab	Two open models serving an **Agent** tab next to the **Trad ML Model** tab
10	Read the fine-tuning run in Experiments and the agent traces	Predictor performance and chat-agent behaviour, side by side
11	Optional: open a VS Code workspace and connect Cline or Codex	An AI assistant that can read and change the project code

1 Get the project ready

Create your project, get the code and the raw data into it, and select the workshop environment. Your administrator has already built the compute environments and hardware tiers (see the admin setup guide), so you only pick them. You can bring the code in from **Git** (recommended) or by uploading a zip into a **Domino-hosted** project. Both end in the same place.

OPTION	CODE COMES FROM	CHOOSE IT WHEN
A. Git	Your own fork of the workshop repository on GitHub, `github.com/DDL-Mike-Snyder/fraud-workshop`	You have a GitHub account and Domino can reach GitHub. Nothing to unzip, and your changes can be committed.
B. Domino-hosted	A code zip that you upload and unzip	No GitHub access, or you were handed the zip files directly.

1a. Create the project

Option A: Git. Do both tasks, then skip option B.

- [ ] **Fork the repository.** On GitHub, open the workshop repository, choose Fork, and create the fork in your own account. If your instructor gave you a different repository URL, use that one.

- [ ] **Create a Git-based project in Domino.** Open Projects, choose New Project, name it, and choose a Git repository as where the code lives. Paste your fork's HTTPS URL, pick the `main` branch, and create the project.

  Domino needs a GitHub credential to read your fork. If it asks, open Account Settings, add a Git credential with a GitHub personal access token that can read the repository, and try again.

Option B: Domino-hosted. Do both tasks, then skip option A.

- [ ] **Create a project.** Open Projects, choose New Project, give it a name and keep the defaults, so the project is hosted in Domino.

- [ ] **Upload the code zip.** Open the project's Code page and upload `fraud-workshop-code.zip` to the project root. You will unzip it below.

1b. Set it up (both options)

- [ ] **Select the workshop environment.** In the project, open Settings and choose `Fraud-Detection-Workshop` as the compute environment.

  If it is not in the list, ask your administrator to check that it is built and visible to you.

- [ ] **Start a JupyterLab workspace** in the project, with the `Fraud-Detection-Workshop` environment, and open a terminal in the project root.

- [ ] **Option B only: unzip the code.** You should then see `exercises/`, `app.sh` and `.setup/` in the file browser. With option A they are already there.

```
unzip -o fraud-workshop-code.zip
```

- [ ] **Put the raw data in the project's dataset.** Open `.setup/unzip_dataset.ipynb` and run it once. It extracts `raw_cc_transactions.csv.gz` to the top level of the dataset. With option A it reads the data zip that came with your code in `support/`. With option B, first open Data, then the project's default Domino Dataset, and upload `fraud-workshop-data.zip` at the top level, then run the notebook.

  Inside workspaces and Jobs, the dataset appears at `/domino/datasets/local/<your-project-name>/` for a Domino-hosted project, and at `/domino/datasets/<your-project-name>/<your-username>/` for a Git-based one. The scripts work this out for you. Confirm the file is listed on the Data page before moving on.

- [ ] **Check the environment from the workspace terminal.** Every line under "Required" must say `ok`.

```
python .setup/verify_environment.py
```

- [ ] **Option B only: sync your workspace files to the project.** Use the workspace's sync control to commit all changes.

  A Domino Job runs the files saved in the project, not the files sitting in an open workspace. If you skip this, the Job in step 2 cannot find `exercises/`. With option A the code is already in the repository, so there is nothing to sync until you change a file. After a change, commit and push it so Jobs and the app see it.

---

2 Prepare the data

The training script reads `transformed_cc_transactions.csv`, and it does not exist yet. Two short runs create it: a notebook that cleans the raw file, then a Job that builds the model features.

2a. Clean the data in a notebook

- [ ] **Start a JupyterLab workspace.** Open Workspaces, create a workspace, choose JupyterLab and the `Fraud-Detection-Workshop` environment, and start it.

- [ ] **Open the notebook and run every cell.** The file is the one below.

```
exercises/a_DataExploration/data_exploration_notebook.ipynb
```

- [ ] **Confirm the result.** The notebook should report 78,324 rows loaded and 1,114 rows dropped (1.42%). Five charts are saved under `artifacts/a_data_exploration/`, and `clean_cc_transactions.csv` appears next to the raw file in the dataset.

  Where to find them. In the JupyterLab file browser on the left (it starts at the project root), open `artifacts`, then `a_data_exploration`, and double-click a file to view it. The five files are `correlation_heatmap.png`, `amount_distribution_by_class.png`, `fraud_rate_by_hour.png`, `top_features_correlation.png` and `categorical_fraud_analysis.png`.

  The same folder appears on the project's Code page, under `artifacts`, once you sync your workspace changes (the next item). In a Domino-hosted project there is no separate Artifacts tab; artifacts are ordinary files in the project's `artifacts` folder. The CSV is not on the Code page: find it under Data, in the project's dataset.

- [ ] **Sync your changes, then stop the workspace.**

2b. Build the model features as a Job

- [ ] **Start a Job.** Open Jobs and choose Run Job. Enter this as the file name or command, keep the workshop environment, pick the Small hardware tier, and start it.

```
python exercises/b_DataEngineering/data_engineering.py
```

- [ ] **Watch it finish.** Expand the Job row to see the status and logs update live. The log should say it loaded 77,210 rows and saved `transformed_cc_transactions.csv`.

- [ ] **Confirm three outputs.** The file `transformed_cc_transactions.csv` is in the dataset; `artifacts/b_data_engineering/preprocessing_report.html` is in the Job's results; and Experiments shows a new experiment starting with "CC Fraud Preprocessing".

WHAT THIS STEP DOES

It adds four columns (the amount compared with the 30-day average, a combined risk score, a trust score, and a generation label from age), scales the number columns and one-hot encodes the text columns. That is why the data grows from 19 columns to 43 (42 inputs and the label).

---

3 Run the model as a Job

Train the XGBoost fraud classifier on the transformed data. Everything it produces is recorded in Experiments.

- [ ] **Start the training Job.** Open Jobs, choose Run Job, enter the command below, and use the Medium hardware tier if your deployment offers it. Give the Job a title such as "Train XGBoost classifier".

```
python exercises/c_TrainingAndEvaluation/trainer_xgb.py
```

Allow several minutes. After training, the script draws plots that refit the model many times (5-fold learning curves), which is the slow part.

- [ ] **Wait for the Job to succeed.** The log ends with "Training XGBoost completed successfully", followed by a small block of numbers: `roc_auc`, `f1_fraud`, `accuracy` and `log_loss`.

- [ ] **Optional: train the other two models for comparison.** Run each as its own Job. You need them only if you want to compare models or serve more than one.

```
python exercises/c_TrainingAndEvaluation/trainer_ada.py
python exercises/c_TrainingAndEvaluation/trainer_gnb.py
```

▼ What the training script does

It reads `transformed_cc_transactions.csv` from the dataset, keeps rows that have a label, and splits them 80% for training and 20% for validation, keeping the same fraud share in both. It trains `XGBClassifier(n_estimators=100, learning_rate=0.1, max_depth=4, subsample=0.8, colsample_bytree=0.8)`.

It records everything in a Domino experiment named "CC Fraud Classifier Training" followed by a short ID for you and your project. The run is named `XGBoost`. Logged items: 3 parameters, 17 metrics, three plots, a parameters file, a metrics file, a JSON result file, and the model itself under the name `xgboost_model`.

4 Inspect the results

Read the numbers and plots to judge the model, and see how Domino keeps the run, code and data together.

- [ ] **Open the Job and read its output.** Check the logs and the results listing. Files written by the script are under `artifacts/`.
- [ ] **Open Experiments.** Find the experiment "CC Fraud Classifier Training..." and open the run named `XGBoost`.
- [ ] **Read the metrics.** Use the table below as a guide.
- [ ] **Open the three plots** in the run's Artifacts: `performance_dashboard.png`, `calibration_plot.png` and `learning_curves_plot.png`.
- [ ] **Look at the run's traceability details**: the code version, environment and hardware it used. This is what lets anyone reproduce the run later.
- [ ] **If you trained all three models**, tick every run in the experiment and choose Compare. In this workshop XGBoost comes out ahead on ROC AUC.

METRIC	HOW TO READ IT
`roc_auc`	How well the model ranks fraud above legitimate transactions. 1.0 is perfect; 0.5 is a coin flip. The headline number for comparing models.
`pr_auc`	The same idea, focused on the fraud class.

METRIC	HOW TO READ IT
`precision_fraud`	Of the transactions flagged as fraud, the share that really were.
`recall_fraud`	Of the real fraud, the share the model caught.
`f1_fraud`	One number that balances precision and recall.
`accuracy`	Share of all transactions classified correctly. About 36% of rows are fraud, so a model that always says "legitimate" would already score near 64%. Do not judge on this alone.
`log_loss`, `brier`	How good the probabilities are, not just the yes/no call. Lower is better.
`tau_best_f1`	The score cutoff that gives the best F1 on the validation data.
`inf_ms_row`	Milliseconds to score one row, useful when thinking about serving.

ONE THING THAT CAN CONFUSE YOU

All three training scripts write plots with the same file names into `artifacts/`, so the last script to run overwrites the earlier ones there. Each run's own copy is safe in its Artifacts tab in Experiments, so look there when comparing models.

5 Register the model

Give the winning model a name and a version so it can be found, governed and deployed.

- [ ] **Open the `XGBoost` run** in Experiments.
- [ ] **Choose Register Model From Run** (top right).

- [ ] **Pick the model artifact `xgboost_model`, enter a name, and register.** Use something specific, such as the name below.

```
fraud-xgboost
```

- [ ] **Open Models** in the project sidebar and confirm your model is listed with version 1 and a link back to the run it came from.

---

6 Publish the model as an endpoint

Turn the registered model into a live REST API. The app in step 7 calls this endpoint to score transactions.

- [ ] **Start a new endpoint from your registered model.** Choose Create endpoint, then ML endpoint. At the top of the window that opens, click the **Create from registered Model** tab, then pick `fraud-xgboost`, version 1.

  The window opens on the **File and Function** tab by default. Do not use that tab: it asks for a Python script, and this project does not have one. If you see a File box, you are on the wrong tab.

- [ ] **Choose the settings.** Use the `Fraud-Detection-Workshop` environment and the small endpoint hardware tier. Leave everything else at its default.

- [ ] **Wait for it to build and start.** It is creating a live serving container, so expect a few minutes.

- [ ] **Copy two values:** the **endpoint URL** from the endpoint's Overview page, and an **access token** that you generate. Keep both handy for step 7.

  The token is not shown on the Overview page. Open the endpoint's **Settings**, then the **Invocation** tab, and generate an access token. Copy it as soon as it appears. A name such as `workshop-app` helps you recognize it later.

  The token is used as both the username and the password (HTTP basic authentication). The gear button next to a token revokes or regenerates it.

IF THE ENDPOINT NEVER STARTS

Open its logs. If you see `no app loaded. GAME OVER` together with `configparser.SafeConfigParser`, the environment was built without the last block of its Dockerfile instructions. Ask your administrator to add it (admin setup guide, step 1), rebuild the environment, and publish the endpoint again.

---

7 Deploy the app

Publish the Streamlit app that sends a transaction to your endpoint and shows the fraud score.

7a. Give the app the endpoint details

- [ ] **Open the project's Settings and add two environment variables.** Set them before you publish the app, because the app reads them when it starts.

NAME	VALUE
`xgboost_endpoint`	The endpoint URL from step 6
`xgboost_auth`	The access token from step 6

The app also looks for `adaboost_endpoint`, `adaboost_auth`, `gaussiannb_endpoint` and `gaussiannb_auth`, for use if you published endpoints for the other two models.

7b. Publish the app

- [ ] **Open Apps & Agents** (under Deployments in the project sidebar, or the Deploy menu), click **Create**, and choose **App from File**.

The menu has three options. Pick **App from File**, which runs a launch script stored in the project. Do not pick **App in Workspace** (it publishes from a live workspace session) or **Agent** (for the optional LLM add-on, not this workshop).

- [ ] **Point it at the launch script.** Use the project files as the source, and enter `app.sh` as the file to run. Choose the `Fraud-Detection-Workshop` environment and the Small hardware tier.

  `app.sh` starts Streamlit on port 8888 and runs `exercises/d_HostingAndExecution/app.py`.

- [ ] **Publish and wait for the app to show as running**, then open it.

The page shows a form with the details of one card transaction: amount, customer age and tenure, risk and trust scores, location, time of day, transaction type, device, merchant category, channel, and a model picker. The app adds the same derived columns you created in step 2, converts the transaction into the 42 input fields the model was trained on, and posts it to your endpoint.

---

8 Check the real model answers

The app has a built-in fallback score, so a page that looks fine can still be skipping the model. Make sure the answer came from your endpoint.

- [ ] **Leave the model picker on "XG Boost"**, fill in a transaction, and choose Predict Fraud Risk.
- [ ] **Read the result.** You get a Fraud Probability and a verdict. Anything above 0.40 is shown as fraud.
- [ ] **Check there is no red error banner.** "Classifier API Error" or "Classifier Connection Error" means the endpoint could not be reached and the number on screen is the fallback.
- [ ] **Confirm in the app's logs.** After a successful call, the log contains a line starting `fraud_prediction =`. That line appears only when the endpoint answered.
- [ ] **Change a few inputs and predict again** to see the score respond.

HOW TO RECOGNIZE THE FALLBACK

The fallback counts how many of six simple rules a transaction trips (amount over 500, merchant risk above 1, device trust below −1, IP reputation below −1, distance above 2, and a night-time hour) and divides by six. Its only possible values are 0.00, 0.17, 0.33, 0.50, 0.67, 0.83 and 1.00.

The "ADA Boost" and "GaussianNB" choices use the fallback unless you published endpoints for those models and set their variables.

---

9 Deploy the two open models

Add an **Agent** tab to the same app. Two open models from Hugging Face run on one GPU: a chat agent that simulates transactions, and a fraud-scoring agent that you fine-tune on this workshop's data.

AGENT	OPEN MODEL	ROLE
Transaction generator	`ibm-granite/granite-3.3-8b-instruct`	Chat with it; it calls a simulator tool to create a batch of transactions that matches what you ask for. Chosen for native tool calling at a size that fits one GPU.
Fraud scorer	`ibm-granite/granite-3.3-2b-instruct`, LoRA fine-tuned	Reads one transaction as text and returns a fraud probability from its next-token probabilities. Small enough to tune in minutes.

MODEL PROVENANCE

Both models are IBM Granite 3.3 (IBM is a US company), released under the Apache 2.0 license and not gated, so they can be mirrored into an air-gapped registry. Models from China-based labs were deliberately excluded for DoD-sensitive deployments. Quality is somewhat lower than the best open models. Confirm with your own approval process before using any model in a DoD environment, since this guide does not certify a model as approved.

GPU SIZE

Both models run on one GPU. On a card with 40 GB or more (for example an L40S 48 GB) the generator runs in full precision. On a 24 GB card `serve_llms.sh` detects the size and loads the generator in 8-bit (FP8) so both models fit, at a small cost in quality. Your administrator provides the GPU hardware tier; GPU tiers can sit in **Pending** for a long time if the cluster has no free GPU node.

9a. Choose the GPU environment

- [ ] **Confirm `Fraud-Detection-LLM` and a GPU hardware tier are available to you.** Your administrator builds the environment (it adds vLLM, transformers and PEFT on top of the workshop environment) and provides the GPU tier. Both appear in the pickers when you start a Job or edit the app. If either is missing, see the admin setup guide.

9b. Fine-tune the fraud model

- [ ] **Make sure you have the open-model code.** It lives in `exercises/f_OpenModelAgents/` and is part of the current code. With option A, pull the latest from your fork if you forked before it was added (on GitHub, choose Sync fork, then pull in the workspace). With option B, upload and unzip the latest code zip as in step 1, then sync your files.

```
git pull
```

- [ ] **Run the fine-tuning Job.** Choose Run Job, the `Fraud-Detection-LLM` environment and your GPU hardware tier, and enter this command.

```
python exercises/f_OpenModelAgents/finetune_fraud_llm.py
```

It downloads the base model into the project dataset, trains a LoRA adapter on 20,000 labelled transactions, scores 2,000 held-out ones, and saves the merged model to `models/fraud-llm` in the dataset. On a 24 GB GPU it took about 25 minutes, including a few minutes to download the model. The log ends with "Fine-tuning completed successfully" and a block of metrics.

9c. Redeploy the app on the GPU

- [ ] **Republish the app from `app.sh` on the GPU.** Open the app from step 7, change its environment to `Fraud-Detection-LLM` and its hardware to the GPU tier, and restart it. Nothing else changes: it is still **App from File** with `app.sh`.

`app.sh` now first runs `serve_llms.sh`, which starts the two models as local servers (generator on port 8000, fraud model on 8001), then starts Streamlit as before. The first start downloads the 7B model, so allow several minutes. If the hardware has no GPU, the servers are skipped and the Trad ML tab still works.

- [ ] **Open the app and check there are two tabs: Trad ML Model** (the step 7 page, unchanged) and **Agent**.

- [ ] **Generate transactions.** In the Agent tab, ask the chat agent for a batch, for example the request below. A table of transactions appears on the right.

```
Make 80 transactions, about 25% fraud, mostly late-night online electronics
purchases far from the cardholder's home.
```

- [ ] **Score them.** Choose Score with fraud agent. Each row gets a `fraud_score` between 0 and 1; move the slider to change what counts as flagged. The page also shows how many of the transactions the generator meant to be fraud were caught.

WHAT WE SAW IN TESTING

On a 24 GB GPU the first start took about 8 minutes because the app downloads the 7B model; later starts were faster because the weights are cached in the project dataset. The app log prints a `SMOKE` and `E2E` check once both models are up. Asking for 40 transactions at 30% fraud produced exactly that, and the fraud agent ranked the simulator's intended fraud above the rest with a ROC AUC of about 0.95. At the default 0.5 cutoff it flagged most of that small batch, so raise the slider if you want fewer flags.

GENERATOR QUALITY

This open 8B model sometimes writes the tool call as code instead of calling the tool. The agent detects that and retries once with the call forced. If the generator still replies with text and no table, rephrase the request more concretely (count, fraud share, and a scenario).

WHAT IS REAL AND WHAT IS NOT

The generator's `is_fraud_intent` column is what the simulator was asked to create, not ground truth. The fraud model was tuned on the workshop's labelled data, so agreement on simulated rows shows the pipeline working, not a validated fraud rate.

---

10 Review the models and traces

The predictor is judged by its metrics in Experiments. The chat agent is judged by its traces, which show each model call and tool call.

10a. Predictor performance

- [ ] **Open Experiments** and find "CC Fraud LLM Fine-tuning...". Open the run named `Granite-LoRA`.

- [ ] **Read the metrics.** `base_roc_auc` is the untuned model; `roc_auc` is after fine-tuning on the same held-out rows. The gap is what fine-tuning bought. In our test run the untuned model scored about 0.53 (a coin flip) and the tuned model about 0.93, with `f1_fraud` about 0.82 and `accuracy` about 0.85 on 2,000 held-out transactions. `f1_fraud`, `precision_fraud`, `recall_fraud` and `tau_best_f1` mean the same as in step 4.

- [ ] **Compare with XGBoost.** Open the "CC Fraud Classifier Training..." experiment and compare its `roc_auc` and `f1_fraud` with the LLM run. A tabular model is often as good or better; the point is to see both on the same data.

- [ ] **Open the run's Artifacts** for the LoRA adapter and `eval_scores.csv` (true label and predicted probability for every held-out row).

10b. Chat-agent traces

- [ ] **Use the Agent tab first.** Send at least two different requests so there is something to look at.

- [ ] **Find the traces.** Open the app's experiment (or Experiments, then the app's traces view, depending on your Domino version) and open a trace named `transaction_generator_agent`.

- [ ] **Expand it.** You should see the model call with your request, the `generate_transactions` tool call with the arguments the model chose (count, fraud rate, scenario), the tool result, and the final answer. Compare the arguments with what you asked for.

- [ ] **Check latency.** Note the time of each span, then open a `fraud_scoring_agent` trace, which records the batch size and flag rate.

---

11 Use a code assistant optional

Work on the project's code with an AI assistant inside a VS Code workspace. You can use **Cline** or **Codex**. Both already know how to talk to Domino, and you bring your own API key. Your administrator sets up the environment (admin setup guide, step 4).

11a. Open a VS Code workspace

- [ ] **Launch a workspace with VS Code.** In the project, choose Workspaces, then Create New Workspace. Pick the `Fraud-Detection-CodeAssist` environment, **VS Code** as the IDE, and the smallest hardware tier that fits, since smaller tiers start faster. If the project asks about Home Directory Persistence, turn it on. Then choose Launch.

The workspace opens in a new browser tab, so allow pop-ups from Domino. If VS Code is not offered, the environment you chose does not include it.

11b. Connect an assistant

Set up one of the two. Do not paste an API key into a chat or commit it to a file.

- [ ] **Cline:** open the Cline panel and choose **Bring my own API key**. For OpenAI, pick the **OpenAI** provider and paste your key. For a company gateway, pick **OpenAI Compatible** and enter the base URL, key and model id your administrator gave you. Choose a model and a reasoning effort, then choose Continue.

Cline's Domino skills and its `domino_server` connection are already in place. Under Cline's MCP Servers, `domino_server` should show as connected, and the first start can take a moment.

- [ ] **Codex:** open a terminal in VS Code (Terminal, then New Terminal) and run the command below. Choose **Sign in with Device Code**, open the address it shows, and enter the code. Then refresh the browser tab and open the Codex tab in the sidebar.

```
codex
```

Sign in through the terminal first, because the extension does nothing until you do. Codex already has the Domino skills.

11c. Try it on this project

- [ ] **Check it can see Domino.** Ask your assistant the question below. Cline should list your clusters and check API access, and Codex should report which Domino user you are connected as.

```
Which Domino user am I connected as, and can you reach the Domino API?
```

- [ ] **Ask it to explain the workshop code.** This one is safe because it only reads.

```
Read exercises/c_TrainingAndEvaluation/trainer_xgb.py and explain, step by
step, what it logs to MLflow.
```

- [ ] **Ask it for a small change, and read the result before you accept it.** For example, in the Agent tab code.

```
In exercises/f_OpenModelAgents/agent_tab.py, add a download button that saves the flagged transactions as a CSV.
```

Sync your files afterwards if you want a Job or the app to pick up the change.

IF SOMETHING IS MISSING

No Cline icon or no Domino skills, or no `domino_server`: restart the workspace, then tell your administrator. If your assistant cannot reach its model, check the base URL and key you entered, or ask your administrator whether the route out of the workspace is open. If Codex does nothing, sign in through the terminal first.

If something fails

WHAT YOU SEE	LIKELY CAUSE	FIX
Job fails with `FileNotFoundError` for `transformed_cc_transactions.csv` or `clean_cc_transactions.csv`	The data steps did not finish	Repeat step 2a, then 2b, and confirm both files are in the dataset
Job cannot find `exercises/...`	Workspace files were never synced to the project	Sync in the workspace, then start the Job again
`No module named 'streamlit'`	Running on the standard environment	Select the workshop environment on the Job, app or project
`No module named 'pkg_resources'` when the EDA report runs	The environment was built without the `setuptools<81` pin	Ask your administrator to use the pip line from step 1 of the admin setup guide and rebuild
Endpoint logs show `no app loaded. GAME OVER`	Missing Python 3.12 fix in the environment	Ask your administrator to add the last Dockerfile block (admin setup guide, step 1), rebuild, republish

WHAT YOU SEE	LIKELY CAUSE	FIX
App shows "Classifier Connection Error: Invalid URL 'None'"	The endpoint variables are not set, or are misspelled	Check both names in project Settings, then restart the app
App shows "Classifier API Error: 401"	Wrong token	Copy the token again; it is used for both username and password
Agent tab says "Could not reach the generator model"	The GPU servers are not up yet, or the app is on a CPU tier or the wrong environment	Check the app is on the GPU tier with `Fraud-Detection-LLM`; wait for the 7B download, and read `/tmp/llm-logs/generator.log` in the app logs
Agent tab says "Could not reach the fraud model"	The fine-tuning Job has not finished, so `models/fraud-llm` does not exist	Finish step 9b, then restart the app
vLLM log shows an out-of-memory error	GPU has less than 40 GB	Use a larger GPU tier, or lower `GEN_GPU_MEM` and use a smaller `GENERATOR_MODEL` such as `Qwen/Granite-3B-Instruct`

Optional extras in the project

The code also includes a Domino Flow that runs all three trainers and compares them (`exercises/c_TrainingAndEvaluation/workflow.py`; it needs a snapshot of the dataset first), a launcher exercise for non-technical users, and an optional LLM agent add-on that needs an API key. None of them is needed for the steps above.

Menu names can differ slightly between Domino versions. If a label here does not match your screen, look for the closest one in the same area.